

1. Introducción:

Este Plan de pruebas está diseñado para definir el alcance, el enfoque, los recursos y el cronograma de todas las actividades de prueba del proyecto Bienestar Universitario.

El plan identifica los elementos que se probarán, las características que se probarán, los tipos de pruebas que se realizará, el personal responsable de las pruebas, los recursos y el cronograma necesarios para completar las pruebas y los riesgos asociados con el plan.

1.1 Alcance

1.1.1 Dentro del Alcance

Todas las funciones de la aplicación Bienestar Universitario que se definieron en los requisitos del software deben ser probadas:

Nombre de la funcionalidad	Roles aplicables	Descripción
Gestión de actividades CADI	Estudiantes, Administradores CADI	Publicación, consulta y actualización de actividades artísticas y deportivas con calendario interactivo y retroalimentación.
Gestión de torneos deportivos y PSU	Estudiantes, Coordinadores	Inscripción digital, creación de equipos, validación de elegibilidad, seguimiento de proyectos sociales y chatbot de asistencia.
Gestión de citas psicológicas	Estudiantes	Reserva, cancelación y reprogramación de citas; visualización de

		disponibilidad en tiempo real.
Gestión de horarios e información	Estudiantes	Calendario personalizado y unificado, notificaciones automáticas y recordatorios.
Analítica y Reportes	Administradores	Consolidación de datos, análisis de comportamiento, generación de métricas, exportación de reportes y registro de asistencia.

1.1.2 Fuera del Alcance

Estas características no se pueden probar porque no están incluidas en las especificaciones de requisitos de software:

- Consolidar datos desde archivos externos (CSV, Excel), bases internas y sistemas externos (Banner, Gestión Humana).

- Limpiar y estandarizar datos (nombres, fechas, formatos), validando estructura y eliminando duplicados.

- Descargar graficas en Excel

- Registrar asistencia por identificación, Excel

- Geolocalización.

- Pruebas de seguridad avanzada y ataques externos.

- Contingencia y sincronización (Módulo 5)

1.2 Objetivo de Calidad

- Verificar el correcto funcionamiento de las funcionalidades principales de la aplicación Bienestar Universitario.

- Asegurar que cada funcionalidad proporcione una experiencia de usuario fluida, intuitiva y sin errores.

- Validar que el sistema responde correctamente ante entradas válidas e inválidas,

manteniendo la integridad de los datos.

- Confirmar que la navegación entre las distintas secciones de la aplicación es consistente, con tiempos de respuesta adecuados.

1.3 Roles y Responsabilidades

No	Rol (Scrum)	Tareas
1	Product Owner	Definir criterios de aceptación en historias de usuario. Priorizar funcionalidades y pruebas en el backlog.
2	Scrum Master	Facilitar la organización del equipo en pruebas. Eliminar impedimentos para la ejecución de pruebas
3	Development Team	Diseñar y ejecutar pruebas unitarias, de integración y aceptación. Reportar defectos. Garantizar calidad continua.
4	Usuarios piloto / Stakeholders	Participar en pruebas de aceptación durante las revisiones de Sprint. Dar retroalimentación de usabilidad.

2. Metodología de prueba

2.1 Niveles de prueba

En el proyecto Bienestar Universitario, se deben realizar 6 tipos de pruebas:

- Pruebas de unidad: Verificación de funcionalidad correcta de los módulos (ej: validación de inscripción en torneos).
- Pruebas de resistencia: Determinar la resistencia del sistema (ej: miles de eventos en calendario).
- Pruebas de regresión: Asegurar que nuevas funciones no rompen funcionalidades existentes.
- Pruebas de desempeño: Simular más de 1000 usuarios concurrentes consultando mapas o inscribiéndose.
- Pruebas de concurrencia: Evaluar cómo maneja múltiples usuarios en la misma funcionalidad simultáneamente.
- Pruebas de volumen: Validar rendimiento con grandes cantidades de datos (más de 50 mil registros).

2.2 Clasificación de errores

Alta: Fallo en funcionalidades críticas (ej: no se pueden reservar citas o inscribir en torneos).

Media: Fallos menores que no bloquean el uso (ej: un mapa que no muestra ubicaciones favoritas).

Baja: Errores visuales o de texto (ej: un nombre de actividad mal escrito).

2.4 Criterios de suspensión y requisitos de reanudación

Si el sistema presenta errores críticos que impiden probar funcionalidades clave, se suspenderán las pruebas hasta que el equipo de desarrollo entregue una nueva versión con correcciones validadas.

2.5 Completitud de la prueba

- Tiempo de respuesta ≤ 2 segundos en búsquedas y consultas.
- Soporte para más de 1000 usuarios concurrentes.
- 95% de los casos de prueba ejecutados exitosamente.

2.6 Tareas del proyecto, estimación y cronograma

Tarea	Miembros	Esfuerzo estimado
Elaborar el plan de pruebas	Product Owner + Development Team	50 Horas-hombre
Diseñar las pruebas	Development Team	100 Horas-hombre
Ejecución de las pruebas	Development Team	70 Horas-hombre
Informe de las pruebas	Development Team + PO	15 Horas-hombre
Corrección y/o aceptación de las pruebas	Development Team + PO	100 Horas-hombre
Entrega de las pruebas	Scrum Master + PO	20 Horas-hombre

Total: 355 Horas-hombre

3. Entregables de prueba

Antes de la fase de prueba:

- Documento de planes de prueba.
- Documento de casos de prueba.
- Especificaciones de diseño de pruebas.

Durante la prueba:

- Registros de ejecución.

- Datos de prueba.
- Reporte de defectos.
- Matriz de trazabilidad de pruebas.

Al finalizar los ciclos de prueba:

- Informe final de pruebas.
- Informe de defectos.
- Manual de instalación/configuración.
- Documentación de la versión.

4. Necesidades de recursos y ambientes

4.1 Herramientas de prueba

Número	Recursos	Descripción
1	Equipos	Teléfono móvil iOS o Android con acceso a internet, cámara y periféricos. PC con Windows 11 o superior, al menos 4GB RAM y procesador Intel i5.
2	Software	OS Android 9.0+, iOS 15.0+, Windows 11+. API de mapas y multimedia.
3	Red	LAN Gigabit y una línea de internet de al menos 5 Mb/s para conectividad multimedia.
4	Herramientas de Prueba	Selenium, JMeter, Postman u otra herramienta que genere resultados automáticos en pruebas.

4.2 Entorno de prueba

Requisitos mínimos de hardware y software que se utilizarán para probar la Aplicación:

- Windows 11 o superior
- Android 9.0 o superior
- iOS 15.0 o superior

Beneficios de las pruebas

Tipo de prueba	Funcionalidad	seguridad	usabilidad	fiabilidad	rendimiento	integridad de datos
Pruebas funcionales	X		X	X		
pruebas de integración	X	X		X		X
pruebas de sistema	X	X		X	X	X
Pruebas de aceptación (UAT)	X		X	X		
Pruebas de seguridad (básica)		X		X		
Pruebas unitarias	X			X		
Pruebas de desempeño				X	X	
Pruebas de datos (in-app)				X		X

Indicadores/Métricas de evaluación de calidad

Métrica	Descripción	Fórmula
Densidad de Errores	Número de errores por unidad de tamaño (documentos o software).	Densidad de errores = N° de errores / Tamaño
Densidad de Defectos	Número de defectos encontrados dividido por el tamaño del software.	Densidad de defectos = N° de defectos / Tamaño
Tasa de Fallos	Tiempo medio entre fallos (<i>Mean Time To Failure, MTTF</i>)..	Tasa de fallos = Tiempo total de operación / N° de fallos

Módulos, Estructura del proyecto.

Carpeta	Descripción funcional	Correspondencia con historias de usuario
Analytics_Reports/	Reportes, análisis, comparaciones, recomendaciones	Módulo 5: <i>Analítica y Reportes</i>
management_CADI/	Gestión de actividades culturales y deportivas	Módulo 1: <i>CADI</i>
tournaments/	Torneos y proyectos sociales (PSU)	Módulo 2: <i>Eventos y Servicios</i>
universityWellbeing/	Núcleo general del sistema (configuración, modelos comunes, calendario, notificaciones)	Módulo 4: <i>Información + Notificaciones</i>

[views.py](#) universidad_WellBeing:

Función / Método	Escenario de Prueba	Datos de Entrada	Resultado Esperado
user_login(request)	Iniciar sesión con credenciales válidas de usuario estudiante	username = "123", password = "abc123"	El sistema autentica al usuario y redirige a la vista home
user_login(request)	Iniciar sesión con credenciales válidas de usuario administrador	username = "admin", password = "admin123"	El sistema autentica al usuario y redirige a la vista cadi_admin
user_login(request)	Iniciar sesión con credenciales incorrectas	username = "123", password = "wrongpass"	Se muestra mensaje "Credenciales incorrectas" y no se inicia sesión
user_login(request)	Usuario sin preferencias accede por primera vez	user sin registro en Preferencias	Se redirige automáticamente a la vista preferences
register(request)	Registrar nuevo usuario con datos válidos	cedula, nombre_completo, email, password válidos	Se crea el registro en User y Participantes, se asocia el rol "Estudiante" y se redirige al login
register(request)	Registrar usuario con correo duplicado	email existente en BD	Se muestra mensaje de error y no se crea el usuario
register(request)	Enviar formulario incompleto (sin nombre o cédula)	datos faltantes	El sistema muestra mensajes de error de validación

preferences(request)	Usuario nuevo selecciona preferencias por primera vez	categorías = [1, 2, 3]	Se crean registros en Preferencias y PreferenciasActividades y redirige a home
preferences(request)	Usuario con preferencias intenta reingresar	user ya con registro en Preferencias	El sistema redirige a home sin duplicar preferencias
home_user(request)	Cargar la página de inicio de usuario con datos completos	user autenticado	Se renderiza home_user.html con actividades, horario y calendario
home_user(request)	Usuario sin preferencias o sin actividades	user sin registros asociados	Se renderiza la plantilla vacía sin errores
get_recommendations_for_user(user)	Usuario con preferencias registradas	user con PreferenciasActividades	Devuelve QuerySet de Actividades relacionadas
get_recommendations_for_user(user)	Usuario sin preferencias registradas	user sin Preferencias	Devuelve lista vacía []
get_user_schedule(user)	Consultar horario del participante	user con registro en HorariosParticipante	Retorna lista de actividades programadas
get_user_calendar(user)	Consultar citas del participante	user con registros en Citas	Retorna lista de citas asociadas al usuario
home_admin(request)	Cargar panel de administrador	user con rol admin	Renderiza la vista home_admin.html correctamente
profile(request)	Consultar perfil de usuario con preferencias	user autenticado	Muestra datos del participante y sus categorías seleccionadas

profile(request)	Consultar perfil sin preferencias registradas	user sin registro en Preferencias	Renderiza vista sin errores, sin lista de actividades
user_logout(request)	Cerrar sesión activa	sesión autenticada	Se cierra la sesión y redirige a login

[Views.py](#) tournaments:

Función / Método	Escenario de Prueba	Datos de Entrada	Resultado Esperado
get_torneo_or_404(id_)	Consultar torneo existente	id_ = torneo válido	Retorna diccionario con datos del torneo y lista de equipos asociados
get_torneo_or_404(id_)	Consultar torneo inexistente	id_ = 99999	Lanza Http404
_parse_date(s)	Fecha válida	"2025-10-01"	Devuelve objeto date(2025, 10, 1)
_parse_date(s)	Fecha inválida o vacía	"abc" o ""	Devuelve None sin lanzar excepción
crear_torneo(request)	Crear torneo con datos válidos	nombre, disciplina_id, fechas válidas	Inserta registro en Torneos, muestra mensaje "Torneo creado correctamente."
crear_torneo(request)	Crear torneo con campos vacíos	nombre vacío	Muestra mensaje "Completa nombre, disciplina y fechas."
crear_torneo(request)	Error al guardar en BD	Datos inválidos o fallo de conexión	Muestra mensaje "No se pudo crear el torneo"
lista_torneos(request)	Listar torneos existentes sin filtro	GET vacío	Renderiza list_tournament.html con todos los torneos

<code>lista_torneos(request)</code>	Filtrar torneos por palabra clave	<code>q = "fútbol"</code>	Devuelve lista filtrada
<code>detalle_torneo(request, id)</code>	Consultar detalle de torneo con equipos	id torneo válido con <code>aforo Equipos</code>	Renderiza plantilla <code>detail_team.html</code>
<code>detalle_torneo(request, id)</code>	Consultar detalle de torneo individual	id torneo sin equipos	Renderiza <code>detail_individual.html</code>
<code>crear_equipo_en_torneo(request, torneo_id)</code>	Crear equipo válido en torneo con cupo	nombre equipo, responsable_id, disciplina_id válidos	Crea registros en Equipos, TorneosEquipos y EquiposParticipantes
<code>crear_equipo_en_torneo(request, torneo_id)</code>	Crear equipo sin nombre o responsable inválido	nombre vacío o responsable inexistente	Muestra mensaje de error y redirige a "create_team"
<code>crear_equipo_en_torneo(request, torneo_id)</code>	Crear equipo en torneo individual	torneo sin <code>aforo Equipos</code>	Muestra mensaje "Este torneo es individual; no permite equipos."
<code>_current_participante(user)</code>	Obtener participante vinculado	user con relación en Participantes	Retorna objeto Participante asociado
<code>_current_participante(user)</code>	Usuario sin participante	user sin relación	Retorna None
<code>unirse_equipo(request, id)</code>	Unirse a equipo válido con cupo	team_id válido, participante no inscrito	Inserta registro en EquiposParticipantes y muestra mensaje de éxito
<code>unirse_equipo(request, id)</code>	Intentar unirse a equipo lleno	miembros = capacidad_max	Muestra mensaje "Este equipo ya alcanzó su capacidad máxima."

<code>unirse_equipo(request, id)</code>	Intentar unirse a equipo de otro torneo	<code>team_id</code> no pertenece al torneo	Muestra mensaje de error y redirige
<code>unirse_equipo(request, id)</code>	Usuario ya inscrito	<code>participante</code> ya en equipo	Muestra mensaje "Ya eres miembro de este equipo."
<code>gestionar_equipo(request, torneo_id, team_id)</code>	Consultar gestión de equipo siendo líder	<code>user</code> = líder del equipo	Renderiza <code>manage_team.html</code> con opciones de gestión
<code>gestionar_equipo(request, torneo_id, team_id)</code>	Miembro intenta eliminar otro sin ser líder	<code>user</code> miembro común	Muestra mensaje "Solo el líder puede quitar a otros integrantes."
<code>gestionar_equipo(request, torneo_id, team_id)</code>	Líder elimina integrante del equipo	<code>remove_id</code> válido	Elimina registro de <code>EquiposParticipantes</code> , muestra "Miembro eliminado."
<code>gestionar_equipo(request, torneo_id, team_id)</code>	Miembro sale del equipo por sí mismo	<code>remove_id</code> = <code>participante</code> actual	Elimina su propio registro y redirige al detalle del torneo
<code>gestionar_equipo(request, torneo_id, team_id)</code>	Intentar eliminar al líder	<code>remove_id</code> = <code>líder_id</code>	Muestra "No puedes quitar al líder del equipo."
<code>inscripcion_individual(request, id)</code>	Inscripción individual con datos válidos	<code>nombre</code> , <code>apellido</code> , <code>cedula(10)</code> , <code>correo</code> válido, <code>género</code>	Retorna vista con <code>ok=True</code>
<code>inscripcion_individual(request, id)</code>	Inscripción individual con correo inválido	<code>correo</code> = "correo@invalido"	Muestra error "Formato de correo inválido"
<code>inscripcion_individual(request, id)</code>	Inscripción individual con	<code>cedula</code> = "12345a"	Error "La cédula debe tener exactamente 10 dígitos"

	cédula corta o no numérica		
inscripcion_individual(request, id)	Intentar inscribir en torneo con equipos	torneo con tiene_equipos=True	Lanza Http404("Este torneo no es individual.")

[views.py](#) Management_CADI

Función / Método	Escenario de Prueba	Datos de Entrada	Resultado Esperado
is_admin(user)	Usuario autenticado con permisos de administrador	user.is_authenticated=True, user.is_staff=True	Retorna True
is_admin(user)	Usuario sin permisos	user.is_authenticated=True, user.is_staff=False	Retorna False
hhmm_to_dt(hhmm)	Hora válida	"15:30"	Devuelve datetime(hoy, 15, 30)
hhmm_to_dt(hhmm)	Hora inválida o nula	"abc" o None	Devuelve None sin excepción
_draft_keys(grupo_actividad_id, actividad_id)	Con actividad_id definido	(1, 2)	Retorna ("cadi_draft_base_1_2", "cadi_draft_sched_1_2")
_draft_keys(grupo_actividad_id, actividad_id)	Sin actividad_id	(3, None)	Retorna ("cadi_draft_base_3_new", "cadi_draft_sched_3_new")
cadi_index(request)	Consultar grupo CADI (id=1) existente	GET	Renderiza listar_grupos_actividades.html con grupo y grupos_actividad

create_Activities(request, ...)	GET inicial sin sesión	GET	Renderiza form_activities.html con listas vacías y modo "create"
create_Activities(request, ...)	POST con acción schedule	action="schedule"	Redirige a vista schedule_draft y guarda datos en sesión
create_Activities(request, ...)	POST con acción confirm y campos vacíos	nombre="", espacio=""	Renderiza form_activities.html con error "Por favor completa Nombre, Espacio y Tipo."
create_Activities(request, ...)	POST con acción confirm y datos válidos	nombre, tipo_id, fechas válidas	Crea Actividades y ActividadesGrupos; redirige a listar_actividades
create_Activities(request, ...)	Error por nombre duplicado	actividad con mismo nombre ya existe	Renderiza form_activities.html con error "Ya existe una actividad con ese nombre."
listar_actividades(request, ...)	Grupo y grupo_actividad válidos	GET	Renderiza listar_actividades.html con lista de actividades
listar_actividades(request, ...)	Slug incorrecto	grupo_nombre ≠ slug real	Redirige a URL con slug correcto
editar_actividad(request, ...)	GET inicial	GET	Carga datos de actividad en form_activities.html modo "edit"
editar_actividad(request, ...)	POST con acción schedule	action="schedule"	Actualiza sesión y redirige a schedule_draft_edit
editar_actividad(request, ...)	POST con acción	nombre vacío	Muestra error "Por favor completa"

	confirm y campos vacíos		Nombre, Espacio y Tipo.”
editar_actividad(request, ...)	POST con tipo inválido	tipo_id="abc"	Muestra error “Tipo de actividad inválido.”
editar_actividad(request, ...)	POST correcto	Datos válidos	Actualiza actividad y redirige a listar_actividades
editar_actividad(request, ...)	Duplicado de nombre	Nombre ya existente	Renderiza con error “Ya existe una actividad con ese nombre.”
listar_grupos_actividad(request, ...)	Grupo existente	GET	Renderiza listar_grupos_actividades.html con descripción contextual
listar_grupos_actividad(request, ...)	Slug incorrecto	grupo_nombre ≠ slug	Redirige a URL corregida
crear_grupo_actividad(request, ...)	GET inicial	GET	Renderiza form_gruposActivi.html
crear_grupo_actividad(request, ...)	POST con datos válidos	nombre, descripción, imagen	Crea GruposActividad y redirige a listar_grupos_actividad
schedule_draft(request, ...)	GET inicial modo “create”	GET	Renderiza schedule.html con campos vacíos y modo “create”
schedule_draft(request, ...)	GET en modo “edit” con actividad_id	GET, actividad existente	Precarga profesor, días y horas desde BD a sesión
schedule_draft(request, ...)	POST crear nuevo horario	profesor, hora_inicio, hora_fin, días	Guarda en sesión y redirige a crear_actividad

schedule_draft(request, ...)	POST en modo "edit"	actividad_id definido	Guarda cambios y redirige a editar_actividad
------------------------------	---------------------	-----------------------	--

[views.py](#) Analytics

Función / Vista	Escenario de Prueba	Datos de Entrada / Condiciones	Resultado Esperado
analytics_index()	Acceso a la vista principal del módulo analítico	GET sin parámetros	Renderiza la plantilla index.html correctamente
analisis_comportamiento()	Filtrar por tipo de actividad	Parámetro tipo_actividad =1	Muestra solo participaciones del tipo seleccionado
analisis_comportamiento()	Filtrar por frecuencia mínima	min_frecuencia =3	Solo se muestran participantes con ≥ 3 participaciones
participantes_list()	Listar todos los participantes	GET sin filtros	Retorna lista ordenada alfabéticamente por nombre
comparaciones()	Agrupar por facultad	agrupacion=facultad	Muestra totales de participaciones por facultad
comparaciones()	Agrupar por género	agrupacion=genero	Totales correctos por género
comparaciones()	Filtrar por semestre	tiempo=semestre, semestre_filtro=3	Solo muestra datos de participantes del semestre 3
comparaciones()	Filtrar por rango de fechas	tiempo=periodo, inicio=2025-01-01, fin=2025-06-01	Se listan participaciones dentro del rango

obtener_estudiantes_activos()	Verificar estudiantes con actividad reciente	Base con asistencias en últimos 14 días	Devuelve lista no vacía con estudiantes activos
obtener_estudiantes_inactivos()	Estudiantes sin asistencias recientes	Ninguna asistencia en últimos 14 días	Lista de estudiantes inactivos
obtener_estudiantes_destacados()	Participantes con ≥ 15 asistencias	BD con registros válidos	Lista de estudiantes destacados
obtener_estudiantes_poca_asistencia()	Participantes con < 3 asistencias	BD simulada	Retorna estudiantes con poca asistencia
obtener_proximos_reconocimientos()	Participantes con 8 o 9 asistencias	BD simulada	Devuelve lista de próximos a reconocimiento
recomendaciones()	Carga de vista principal	GET sin parámetros	Renderiza recomendaciones.html con todas las listas contextuales
recomendaciones()	Envío de notificaciones activado	GET con ?enviar_notificaciones=1	Mensaje de éxito y correos registrados en log
log_email()	Registro de correos enviados	asunto="Test", destinatarios=["a@b.com"]	Crea línea en log_emails.txt
enviar_email()	Envío correcto de correo	Datos válidos y settings.DEFAULT_FROM_EMAIL configurado	Retorna sin error y guarda en log
gestion_asistencia()	Listar asistencias por fecha	fecha=YYYY-MM-DD	Renderiza gestion_asistencia.html con registros del día

gestion_asistencia()	Filtrar por actividad	actividad=1	Solo muestra asistencias de la actividad seleccionada
gestion_asistencia()	Filtrar por participante	participante="Juan"	Muestra solo coincidencias de nombre
gestion_asistencia()	Filtrar por estado	estado=Presente	Muestra solo asistencias con ese estado
obtener_participantes_actividad()	Petición con ID válido	actividad_id=1	Devuelve JSON con participantes relacionados
obtener_participantes_actividad()	Sin ID	No se envía actividad_id	Retorna error 400 JSON
historial_participante()	Consultar historial existente	participante_id=1	Muestra historial con asistencias y notas recientes
asistencia()	Generar resumen general	GET sin parámetros	Renderiza asistencia.html con conteo por estado
registrar_asistencia_cedula_view()	Cargar vista de registro rápido	GET	Renderiza formulario con lista de actividades y estados
registrar_asistencia_rapido()	Registrar nueva asistencia	POST con cedula, actividad_id, estado_id, fecha válidos	Retorna JSON {success: True}
registrar_asistencia_rapido()	Actualizar asistencia existente	Mismo participante y fecha	Actualiza estado existente correctamente
registrar_asistencia_rapido()	Faltan campos	POST incompleto	Devuelve JSON de error 400
registrar_asistencia_manual()	Registrar múltiples asistencias válidas	Lista de cédulas válidas	Crea usuarios, participaciones y asistencias

registrar_asistencia_manual()	Repetir cédula existente	Cédula duplicada en texto	Muestra advertencia “ya tiene asistencia registrada”
enviar_notificaciones_automaticas()	Ejecución con contextos válidos	Diccionario con listas no vacías	Llama a funciones de envío correspondientes
enviar_alerta_poca_asistencia()	Lista de estudiantes con baja asistencia	Datos simulados	Envío de correo al staff
enviar_notificacion_reconocimientos()	Lista de estudiantes destacados	Datos simulados con correos	Envía correo personalizado a cada participante
enviar_alerta_inactividad()	Lista de inactivos	Datos simulados	Envía correo al staff
configurar_notificaciones()	Guardar configuración válida	POST con valores de formulario	Muestra mensaje de éxito
generar_encuesta_feedback()	Estudiantes con actividades finalizadas	BD con fecha_finalizacion reciente	Llama a enviar_encuesta_feedback() para cada uno
enviar_encuesta_feedback()	Envío individual	Participación válida con correo	Envía correo con link de Google Forms